

# Module 8: Spring Boot Production Deployment — AWS, Profiles, CI/CD, and Flyway

## Questions

### Q1. What are the main things to consider when moving a Spring Boot application from local development to production?

**Answer:** Production readiness is much more than deploying a JAR file. You need a managed database with backups, environment-specific configuration, secrets management, health monitoring, centralized logging, and a repeatable deployment pipeline. On AWS this usually means RDS for the database, Elastic Beanstalk or ECS/EKS for compute, a load balancer for traffic distribution, and CloudWatch for metrics and alarms. You also need to externalize configuration using Spring profiles and environment variables so the same artifact runs in dev, staging, and production without code changes. A common pitfall is shipping the production database password inside the JAR or leaving debug endpoints exposed.

---

### Q2. How do you set up a production database with Amazon RDS?

**Answer:** I create a DB subnet group that spans at least two availability zones for high availability, then provision the RDS instance or Aurora cluster with the desired engine, instance class, storage, and backup retention. Security groups restrict inbound traffic to the application tier only, and the database is placed in private subnets so it is unreachable from the internet. I enable automated backups, multi-AZ failover for production workloads, and encryption at rest using KMS. The application connects through an environment variable such as `SPRING_DATASOURCE_URL`, which points to the RDS endpoint, rather than hard-coding the host. I also create a read replica if reporting queries need to offload traffic from the primary instance.

---

### Q3. What is the difference between RDS Multi-AZ and a read replica?

**Answer:** Multi-AZ maintains a synchronous standby in another availability zone for high availability. If the primary fails, RDS automatically fails over to the standby without manual intervention. It is for availability, not scaling. A read replica is an asynchronous copy of the primary that can serve read traffic and be promoted to a standalone database. It is for scaling read-heavy workloads and disaster recovery, but it may lag behind the primary by a small amount. Teams sometimes confuse the two and enable Multi-AZ expecting read scaling, or create a read replica expecting automatic failover, so it is important to choose based on the actual goal.

---

### Q4. How do you manage database credentials securely in a Spring Boot application on AWS?

**Answer:** Credentials should never be committed to source control or baked into the Docker image. In AWS, the cleanest approach is to store the username and password in AWS Secrets Manager and

have the application retrieve them at startup, either by integrating Spring Cloud AWS or by mounting the secret as environment variables through Elastic Beanstalk or ECS. Another option is to use IAM database authentication for supported engines, which issues short-lived tokens and removes long-lived passwords. In a pinch you can pass the secret as environment variables, but rotation then becomes manual. Whatever the mechanism, the application must fail clearly at startup if the secret cannot be read, so misconfiguration is discovered immediately rather than masked by a connection error hours later.

---

## **Q5. What are Spring profiles, and how do they help with environment-specific configuration?**

**Answer:** Spring profiles let you define sets of beans and properties that apply only in specific environments. You activate a profile with `spring.profiles.active=prod`, and Spring loads `application-prod.properties` or `application-prod.yml` alongside the main configuration. Profiles are useful for switching datasources, logging levels, feature flags, and external service URLs. The real value is that the same deployable artifact can run anywhere. A common mistake is putting environment-specific values directly in `application.properties`; instead, put defaults there and override them per profile or through environment variables so the artifact remains portable.

---

## **Q6. How do you structure `application.properties` or `application.yml` for dev, staging, and production?**

**Answer:** I keep common settings in the main `application.yml`, such as application name, Jackson settings, and logging format. I then create profile-specific files like `application-dev.yml`, `application-staging.yml`, and `application-prod.yml` that contain environment-specific overrides. Sensitive values such as passwords are not stored in any file; they are supplied through environment variables or a secret manager. For example, `spring.datasource.url` can point to a local Docker database in dev and an RDS endpoint in prod. I also use a `local` profile for developers that disables cloud integrations and uses an in-memory or Dockerized database.

---

## **Q7. What is the `@Profile` annotation, and when would you use it?**

**Answer:** `@Profile` lets you register a bean only when a specific profile is active. For example, you might have a `MockNotificationService` annotated with `@Profile("local")` and a real `AwsSesNotificationService` annotated with `@Profile("!local")`. This keeps the application runnable locally without external credentials. It is also useful for feature flags during gradual rollouts. The risk is overusing profiles to toggle behavior; if every environment behaves differently, the production path is not well tested. Profiles should mainly handle externalized configuration and infrastructure wiring, not business logic branches.

---

## Q8. What is AWS Elastic Beanstalk, and why is it a common choice for Spring Boot deployments?

**Answer:** Elastic Beanstalk is a managed platform service that handles capacity provisioning, load balancing, auto-scaling, and application health monitoring for you. You upload your JAR or Docker image, choose a platform such as Java 17, and Elastic Beanstalk deploys it onto EC2 instances behind a load balancer. It is a common choice for Spring Boot because it reduces operational overhead; you get a production-grade environment without manually configuring EC2, load balancers, and scaling policies. The trade-off is less control than ECS, EKS, or raw EC2, and some teams outgrow it as they need custom networking or advanced deployment strategies.

---

## Q9. How do you deploy a Spring Boot JAR to Elastic Beanstalk with a load balancer and auto-scaling?

**Answer:** I create an Elastic Beanstalk web server environment and choose a load-balanced configuration. The environment provisions an Application Load Balancer, an Auto Scaling group, and EC2 instances running the Java platform. I upload the Spring Boot JAR as an application version and deploy it to the environment. In the environment configuration, I set health check URL to `/actuator/health`, configure auto-scaling triggers based on CPU or request count, and set instance type and minimum/maximum counts. The load balancer forwards traffic on port 80 or 443 to the instances on the port Spring Boot listens on, usually 5000 for Elastic Beanstalk Java platforms.

---

## Q10. What port does a Spring Boot application run on when deployed to Elastic Beanstalk, and how do you configure it?

**Answer:** Elastic Beanstalk Java platforms default to running the application on port 5000, and the nginx proxy forwards requests from port 80 to port 5000. If your Spring Boot application uses a different port, you must set `SERVER_PORT` to 5000 or update the nginx configuration. The platform can pass the port through the `PORT` environment variable. A common issue is that the application starts on 8080 locally and fails health checks on Elastic Beanstalk because the load balancer expects it on 5000. I always verify the port setting in the environment properties and use `server.port=5000` or the equivalent environment variable.

---

## Q11. How do you connect an Elastic Beanstalk application to an RDS database?

**Answer:** I create the RDS instance first and then configure the Elastic Beanstalk environment properties to point to it. The properties include `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, and `SPRING_DATASOURCE_PASSWORD`. The security group of the RDS instance must allow inbound traffic from the security group attached to the Elastic Beanstalk EC2 instances. It is best not to use Elastic Beanstalk's integrated RDS option for production because it ties the database lifecycle to the environment; instead, create RDS separately and reference it through environment properties. This keeps the database alive if the environment is

recreated or scaled down.

---

## **Q12. What is auto-scaling, and what metrics should trigger it in a Spring Boot application?**

**Answer:** Auto-scaling automatically adds or removes EC2 instances based on demand to maintain performance and control cost. Common triggers are CPU utilization above a threshold for several minutes, request count per target, network traffic, or custom CloudWatch metrics such as queue depth. For Spring Boot APIs, CPU is the simplest starting point, but request count can scale faster for IO-bound workloads. Scaling policies should have cooldowns so instances are not added and removed too quickly. I also configure a minimum instance count for availability and a maximum to prevent runaway costs during traffic spikes or attacks.

---

## **Q13. What is CI/CD, and how does AWS CodePipeline and CodeBuild help?**

**Answer:** CI/CD is the practice of automatically building, testing, and deploying code changes. AWS CodeBuild compiles the code, runs tests, and produces artifacts such as a JAR or Docker image. AWS CodePipeline orchestrates the stages: source from GitHub, build with CodeBuild, and deploy to Elastic Beanstalk, ECS, or S3. GitHub hooks trigger the pipeline whenever code is pushed, so there is no manual handoff. The benefit is fast, repeatable, and auditable releases. A common mistake is skipping tests in the pipeline or deploying directly to production without a staging stage, which defeats the purpose of having a pipeline.

---

## **Q14. How do you configure GitHub webhooks to trigger AWS CodePipeline?**

**Answer:** In CodePipeline, I set the source provider to GitHub version 2, which uses a GitHub App connection to authenticate. During setup, CodePipeline creates a webhook in the GitHub repository that sends push events to the pipeline. Alternatively, you can use a webhook created through API Gateway or a Lambda function that starts the pipeline. The webhook payload is filtered by branch, so only pushes to main or release branches trigger a build. I also ensure the pipeline has a manual approval stage between staging and production so a human can gate the final deployment.

---

## **Q15. What buildspec file does CodeBuild need for a Spring Boot Maven project?**

**Answer:** CodeBuild uses a `buildspec.yml` file to define phases and artifacts. The phases typically include install, pre-build, build, and post-build. In the build phase, I run `mvn clean verify` to compile, run unit and integration tests, and package the JAR. In post-build, I copy the JAR to the artifacts directory. The artifacts section tells CodeBuild which files to pass to the next pipeline stage, such as `target/*.jar` and an `application.properties` if needed. I also cache the Maven dependencies in `~/.m2` to speed up subsequent builds. If tests fail, the build stops

and nothing is deployed.

---

### **Q16. What is database migration, and why is it important in production?**

**Answer:** Database migration is the controlled process of applying schema changes such as adding tables, columns, indexes, or constraints as the application evolves. In production, manual changes are risky because they are hard to reproduce, review, and roll back. Migration tools such as Flyway or Liquibase version schema changes as scripts and apply them automatically in a defined order. This makes deployments repeatable across environments and keeps the schema in sync with the code. A production incident often happens when a code change expects a column that was never added to the production schema.

---

### **Q17. How does Flyway work in a Spring Boot application?**

**Answer:** Flyway is a database migration tool that applies SQL or Java migration scripts stored in `db/migration`. Scripts are named with a version prefix such as `V1__create_users_table.sql`, and Flyway executes them in order. It maintains a `flyway_schema_history` table to track which migrations have already been applied, so it never runs the same script twice. In Spring Boot, adding `flyway-core` to the classpath enables auto-configuration that runs migrations at startup. This means the application will not start if a migration fails, which prevents running code against an incompatible schema.

---

### **Q18. What is the difference between versioned migrations and repeatable migrations in Flyway?**

**Answer:** Versioned migrations are numbered scripts that run exactly once and are used for schema changes that must not be repeated, such as adding a column or creating a table. Repeatable migrations have a description but no version number, and they run whenever their checksum changes. They are useful for views, stored procedures, functions, and reference data that should be recreated when the script is updated. A typical setup uses versioned migrations for structural changes and repeatable migrations for database logic that depends on those structures.

---

### **Q19. How do you handle destructive changes in a Flyway migration safely?**

**Answer:** Destructive changes such as dropping a column or table should be done in multiple deployments to avoid data loss and downtime. Instead of dropping a column immediately, I first deploy a migration that adds a new column and copies data, update the application to read from both, then deploy another migration that removes the old column once the application no longer uses it. Flyway's versioning guarantees order, but it does not protect you from the business impact of a bad script. I always review migrations in a pull request, test them against a copy of production data, and have a rollback plan such as restoring from a snapshot.

---

## Q20. How do you manage environment-specific Flyway configuration in Spring Boot?

**Answer:** Flyway reads properties from the active Spring profile, so you can have different schemas, locations, or baseline settings per environment. For example, `application-prod.yml` might set `spring.flyway.locations=classpath:db/migration,classpath:db/migration/prod` and disable clean so the production database is never wiped. In dev, you might enable `spring.flyway.clean-disabled=false` and use a Dockerized database. Sensitive connection details come from environment variables or AWS Secrets Manager, not from the Flyway configuration file. I also set `spring.flyway.baseline-on-migrate=true` when introducing Flyway to an existing database that already has a schema.

---

## Q21. What is the difference between blue-green deployment and rolling deployment on Elastic Beanstalk?

**Answer:** A rolling deployment updates instances in place a few at a time, which is simple but means old and new versions run simultaneously during the transition. A blue-green deployment creates a separate environment with the new version, verifies it, and then swaps the load balancer to point to the new environment. Blue-green deployments are safer because you can rollback by swapping back, but they cost more because both environments run briefly. Elastic Beanstalk supports swap URLs for blue-green deployments, while rolling updates are a built-in deployment policy.

---

## Q22. How do you monitor a Spring Boot application deployed on AWS?

**Answer:** I expose health, metrics, and info endpoints through Spring Boot Actuator and secure them so only authorized systems can read them. CloudWatch can collect logs from the Elastic Beanstalk environment and trigger alarms on error counts, CPU, or memory. For application-level metrics, I use Micrometer to publish custom metrics such as request duration, order creation count, or payment failure rate to CloudWatch or Prometheus. I also set up SNS notifications so the team is alerted when alarms fire. Health checks on the load balancer should use `/actuator/health` so unhealthy instances are replaced automatically.

---

## Q23. How do you handle log aggregation for distributed Spring Boot instances?

**Answer:** In Elastic Beanstalk, instances come and go, so logs stored locally are lost when an instance is terminated. I configure the environment to stream logs to CloudWatch Logs using the Elastic Beanstalk log streaming option, or I use the CloudWatch agent inside Docker deployments. The application should write structured JSON logs with correlation IDs so log entries can be traced across services. For larger systems, I forward logs to a centralized tool such as ELK, Splunk, or Datadog. I also ensure that sensitive data is not logged, and that log levels are configurable per profile.

---

## Q24. What is infrastructure as code, and why is it useful for AWS deployments?

**Answer:** Infrastructure as code is the practice of defining AWS resources such as RDS, Elastic Beanstalk, load balancers, and IAM roles in declarative templates using tools like Terraform, AWS CloudFormation, or the AWS CDK. It makes deployments repeatable, version-controlled, and reviewable. If a production environment is hand-built, recreating it after an incident requires guessing what was configured. With infrastructure as code, you recreate the environment by running the same template. It also supports separate environments that are identical except for size and secrets, which reduces "works in dev, fails in prod" issues.

---

## Scenario based questions

### S1. A production deployment fails because the application cannot connect to RDS after switching from dev. How do you investigate?

**Answer:** I begin by reading the application startup logs carefully, because the exact exception tells me whether this is a network reachability problem or an authentication problem. A connection timeout usually means the security group is blocking traffic, the RDS instance is in a different VPC, or the subnet routing is wrong, while an authentication failure means the username, password, or database name is incorrect. I verify the environment variables in Elastic Beanstalk or ECS match the values shown in the RDS console, and I confirm that the RDS security group has an inbound rule allowing the application security group on the database port. I also check that the RDS instance is in an available state and that the subnet group spans availability zones compatible with the application. If the database was recently restored or recreated, the endpoint will have changed and the application configuration must be updated before the next deployment. I avoid the trap of trying different credentials blindly or opening the security group to the internet, because both increase risk and do not fix a routing problem.

---

### S2. A Spring Boot application works in dev but uses the dev database in production because of hard-coded configuration. How do you fix the configuration strategy?

**Answer:** I first search the codebase for any hard-coded localhost URLs, default usernames, or passwords that are checked into the repository, because these are the values the application will use if no environment override is present. I move all dev-only values out of `application.properties` and into `application-dev.yml`, and I make the main configuration file contain only safe defaults or placeholders. For production, I set `SPRING_PROFILES_ACTIVE=prod` and supply `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, and `SPRING_DATASOURCE_PASSWORD` through Elastic Beanstalk environment properties or, better, AWS Secrets Manager. I validate the fix by running the application locally with the `prod` profile and dummy credentials to confirm it attempts to connect to the supplied URL rather than a hard-coded dev database. I also add a CI step that scans

the repository for hard-coded secrets and localhost database URLs, failing the build if any are found. The lesson is that the same artifact should run everywhere, with environment-specific values injected at runtime rather than baked into the build.

---

### **S3. A deployment to Elastic Beanstalk succeeds, but health checks fail and the environment rolls back. What do you check?**

**Answer:** I open the Elastic Beanstalk events and the application logs in CloudWatch immediately, because a successful deployment only means the files were copied, not that the application is healthy. The most common reasons for failed health checks are the application listening on the wrong port, the health endpoint returning a non-200 status, or the application crashing during startup because it cannot reach the database or apply migrations. I verify that the platform expects the application on port 5000 and that `SERVER_PORT` is set accordingly. I check that `/actuator/health` is accessible anonymously or that the load balancer health check path is configured to a public endpoint. If Spring Security is blocking the health check, I permit access to `/actuator/health` and `/actuator/info` in the security configuration. I also verify that Flyway migrations run successfully at startup, because a failed migration can leave the application unable to start and therefore unable to pass health checks. Once the cause is fixed, I deploy again and monitor the health dashboard before declaring the release successful.

---

### **S4. A Flyway migration runs successfully in dev but fails in production because a table already exists. How do you resolve it?**

**Answer:** This typically happens when Flyway is introduced to a database that was already created and modified manually, so the production schema is at a later state than the first migration script assumes. I start by checking the `flyway_schema_history` table, or its absence, to understand what Flyway thinks has been applied. I then baseline the production database by setting `spring.flyway.baseline-on-migrate=true` and choosing a baseline version that matches the current schema state, which tells Flyway to mark earlier migrations as applied without running them. After baselining, all future migrations must be written against the current production schema. I never manually edit the `flyway_schema_history` table to force a migration through, because that corrupts Flyway's internal state and leads to unpredictable failures later. Before applying the baseline in production, I restore a snapshot to a staging environment, run the baseline and the planned migrations there, and run the application smoke tests to confirm everything works. The long-term fix is to ensure every environment is created from the same migration sequence so manual drift does not happen again.

---

### **S5. A CI/CD pipeline deploys to production automatically whenever code is pushed to main. After a bad release, the team wants a staging gate. How do you redesign the pipeline?**

**Answer:** I restructure the pipeline into clear stages: source, build, staging, manual approval, and production. In the build stage, CodeBuild compiles the code, runs unit and integration tests, and



produces the deployable artifact. In the staging stage, the artifact is deployed to a staging Elastic Beanstalk environment that mirrors production as closely as possible, and a set of automated smoke tests and integration tests run against it. I then add a manual approval action in CodePipeline that requires a team member to approve the deployment after reviewing the test results, the change log, and any monitoring anomalies. For production, I keep the previous application version available and use blue-green deployment so that if the new version fails health checks, the load balancer can be swapped back quickly. I also add GitHub branch protection rules requiring pull request reviews before code can reach main, so the pipeline is not triggered by unreviewed changes. The key principle is that no change should reach customers without being exercised in a staging environment first.

---

## **S6. A migration script drops an unused column, and the rollback requires restoring from backup. How do you prevent this next time?**

**Answer:** This is a classic destructive migration mistake, and the immediate lesson is that dropping a column should always be the final step of a multi-release process. The first step is to remove every application reference to the column and deploy that change. The second step is to verify that no code, report, or downstream consumer reads from or writes to the column. Only then do I write a migration that drops it, and before applying that migration I take a manual snapshot of the production database. I also run the migration against a restored copy of production data in staging to confirm it behaves as expected. For columns that might contain useful historical data, I copy the data to an archive table or a data lake before removal. In the pull request, I label the migration as destructive and require two approvals. If the column was already dropped and data is needed, I restore the pre-drop snapshot to a separate instance and extract the data, rather than trying to undo the migration by hand.

---

## **S7. Auto-scaling adds instances during peak hours but costs spike unexpectedly. How do you balance performance and cost?**

**Answer:** I begin by reviewing the auto-scaling history and CloudWatch metrics to see exactly when and why instances were added. CPU is the default trigger, but for an IO-bound Spring Boot application the CPU may stay low while response time degrades, so scaling based on CPU alone can add instances too late or unnecessarily. I consider switching the scaling metric to request count per target, average response time, or a custom CloudWatch metric such as queue depth. I set a maximum instance count as a hard cost ceiling and use scheduled scaling to reduce capacity during known quiet periods. I also examine whether the application can handle more load per instance by tuning the JVM heap, database connection pool size, or slow queries, because adding instances is not always the cheapest fix. For predictable baseline load, I use Reserved Instances or Savings Plans to reduce the per-hour cost, and I reserve Spot instances only for stateless, fault-tolerant batch work. The caution is that aggressive cost optimization should never remove the minimum instances needed for availability.

---

## **S8. A load balancer is distributing traffic, but one instance is unhealthy and still receiving requests. Why?**

**Answer:** I first check the health check configuration on the target group, because an incorrect path, a long interval, or a high number of healthy thresholds can delay the load balancer from detecting a failure. If the health check path is a static page that always returns 200 even when the application cannot reach the database, the load balancer will continue sending traffic to a broken instance. I change the health check to `/actuator/health` or another endpoint that genuinely reflects application readiness. Even with a correct health check, there is a short window between the first failed check and deregistration when the instance can still receive requests, and connection draining or deregistration delay lengthens that window. I tune these values to balance fast failure detection with graceful shutdown of in-flight requests. I also enable the load balancer access logs and target group metrics so I can see exactly when an instance was marked unhealthy and how long it remained in rotation.

---

## **S9. A developer checks AWS access keys into the GitHub repository. What do you do immediately?**

**Answer:** I treat the keys as compromised the moment they touch version control, even if they were removed quickly, because Git history retains them. I immediately rotate the credentials in the AWS IAM console by deactivating and deleting the exposed access keys and creating new ones for the affected user or service. I then inspect CloudTrail logs for any unauthorized API calls made with the old keys, paying attention to unusual regions, resource creation, or data access. I run GitHub secret scanning or `git-secrets` across the repository history to find any other exposed credentials, and if necessary I rewrite history to remove them. I add pre-commit hooks and repository scanning to prevent future commits of secrets, and I update the team's onboarding documentation to explain why access keys must be provided through environment variables or IAM roles rather than code. I also recommend switching from long-term access keys to IAM roles wherever possible, because roles do not expose static credentials.

---

## **S10. The team wants to switch from Elastic Beanstalk to containers on ECS. What changes in the deployment pipeline?**

**Answer:** The biggest change is that the build artifact becomes a Docker image rather than an executable JAR. In CodeBuild, I update the `buildspec.yml` to run `docker build`, tag the image with the commit hash or build ID, and push it to Amazon ECR. The deploy stage then uses ECS task definitions and service updates instead of Elastic Beanstalk application versions, so the pipeline must generate or update a task definition with the new image URI and call `ecs update-service`. Load balancing shifts to an Application Load Balancer that points at an ECS target group, and auto-scaling becomes ECS service auto-scaling. Environment configuration is still passed through environment variables, but in ECS these live in the task definition or are sourced from AWS Secrets Manager and Systems Manager Parameter Store. I also add container health checks and CloudWatch logging sidecars so observability is consistent with the previous setup. The transition requires the team to understand container networking, task lifecycle, and image versioning, but it gives more flexibility for blue-green and canary deployments.

---

**S11. A new environment variable is added to the application, but production crashes on startup because it is missing. How do you prevent this in the pipeline?**

**Answer:** I add a validation step early in the pipeline that compares the environment configuration against a required manifest. This can be a simple script in CodeBuild that reads a `required-env.properties` file and checks that every listed variable is present in the target environment before deployment proceeds. In the Spring Boot application, I make the critical property required using constructor binding with validation annotations or by implementing an `ApplicationRunner` that fails fast with a clear message if a required value is missing. I keep environment-specific configuration documented in version control, ideally as Terraform or CloudFormation templates, so the infrastructure and the application expectations stay in sync. I also run the application with the `prod` profile and minimal environment variables in a CI smoke test to catch missing configuration before it reaches production. The broader lesson is that configuration changes should be reviewed and applied through the same pipeline as code changes.

---

**S12. A migration fails because two developers wrote scripts with the same version number. How do you resolve the conflict?**

**Answer:** Flyway requires every versioned migration to have a unique prefix, so two files named `V5___` . . . will cause `validate` to fail. I coordinate with the other developer to decide which script logically comes first, then I rename one script to the next available version number and make sure the ordering still makes sense from a database state perspective. If either script was already applied to a shared environment, I check the `flyway_schema_history` table before renaming, because renaming an applied script will change its checksum and cause validation errors. To prevent this from recurring, I introduce a naming convention based on ticket numbers or the current date, and I add a CI step that runs `flyway validate` or checks for duplicate version prefixes before the pull request can be merged. I also encourage the team to announce when they are adding migrations on shared branches so two people do not independently pick the same version number.

---

**S13. The application logs are full of stack traces, but the team cannot find the original error quickly. How do you improve observability?**

**Answer:** I start by adding a correlation ID at the entry point of every request, usually through a servlet filter or interceptor, and I propagate that ID through the MDC so that every log statement emitted during the request includes the same identifier. I then change the log format to structured JSON with fields such as timestamp, level, logger, message, correlation ID, request path, and user ID, and I send those logs to CloudWatch Logs. This lets the team use CloudWatch Logs Insights to query all log lines for a single request in seconds. I also classify errors more carefully: business validation failures return a clear 4xx response with a short message and log at `WARN`, while unexpected internal errors log the full stack trace at `ERROR` and return only a generic incident message to the client. I make sure sensitive data such as passwords or tokens are never logged, and I add alerts on `ERROR` rate spikes so the team knows about problems before users report them.

---

### **S14. A production deployment happens during business hours and causes downtime. How do you move to zero-downtime deployments?**

**Answer:** I first move the deployment window outside business hours as a short-term measure, but the real fix is to change the deployment strategy. In Elastic Beanstalk, I use blue-green deployment by cloning the environment, deploying the new version, running smoke tests, and swapping the load balancer URL to point to the new environment; if anything goes wrong, I swap back. In ECS, I configure a rolling update with minimum healthy percent of 100 and maximum percent of 200 so new containers start and pass health checks before old containers stop. For higher confidence, I add a canary stage that routes a small percentage of traffic to the new version while monitoring error rates and latency. All database migrations are applied before the code deployment and must be backward-compatible, meaning old code can still run against the new schema until the deployment finishes. I also verify that the load balancer health checks are strict enough to prevent traffic from reaching instances that have started but are not yet ready.

---

### **S15. A staging deployment uses the same RDS instance as production to save cost. What are the risks?**

**Answer:** Sharing a database between staging and production is one of the riskiest cost optimizations because it breaks environment isolation completely. A bug in a staging feature can corrupt or delete production data, a load test can starve production queries and cause timeouts for real users, and a schema migration tested in staging is applied directly to the production schema. There is also no safe way to test destructive changes or data fixes without affecting production. The right approach is to use a separate RDS instance or Aurora cluster for staging, ideally restored periodically from a sanitized copy of production data so tests run against realistic schema and volumes. If budget is tight, I use a smaller instance class for staging, schedule it to stop during nights and weekends, or use RDS snapshots to recreate it on demand. The database is the one resource where the cost of sharing always outweighs the savings.

---

### **S16. The RDS storage usage is growing rapidly and approaching the maximum provisioned size. How do you respond?**

**Answer:** I first identify what is consuming storage by checking CloudWatch metrics for database size growth, transaction logs, and temporary files. If the growth comes from normal data accumulation, I enable RDS storage auto-scaling so the volume grows automatically up to a defined maximum without manual intervention. If storage auto-scaling is not available for the engine or instance class, I plan a maintenance window to increase the allocated storage manually. If the growth is caused by excessive transaction logs, I review the backup retention period and log expiration settings, and I make sure long-running transactions or replication lag are not preventing log cleanup. For audit or historical data that does not need to be in the primary database, I archive old rows to S3 or a data warehouse and purge them in batches. I avoid filling the disk to 100 percent, because many databases become read-only or crash at that point, and I set CloudWatch alarms at 70 and 85 percent so the team has time to act.

---

### **S17. HTTPS requests to the Elastic Beanstalk environment fail with a certificate error. How do you fix it?**

**Answer:** I check whether the Application Load Balancer or the instance itself is terminating TLS. In most Elastic Beanstalk setups, the load balancer terminates HTTPS using an ACM certificate, and the instances handle HTTP. I open the load balancer listener configuration and verify that port 443 is present, that it has a valid ACM certificate attached, and that the certificate covers the domain being requested. If the certificate has expired, I renew it in ACM and update the listener. If the application is accessed directly by IP or an old DNS name that is not on the certificate, the browser will show a name mismatch error, so I make sure the DNS record points to the load balancer's canonical name. I also confirm that the security group allows inbound HTTPS traffic on 443. I avoid putting self-signed certificates on the instances unless absolutely necessary, because ACM provides free managed certificates that renew automatically.

---

### **S18. The CodeBuild project builds successfully on a developer's laptop, but the same tests fail in the pipeline. What do you check?**

**Answer:** I start by reading the CodeBuild log to see exactly which tests failed and how they failed, because environment differences are the most common cause. The pipeline may run tests against a different database profile, a different timezone, or missing environment variables that are present on the developer's machine. I check the `buildspec.yml` to confirm that the correct Maven profile is active and that any required services or resources are available during the build. I also verify that the build does not rely on files that are in the developer's local workspace but not in the repository, such as untracked configuration files. Differences in Java version, Maven version, or locale between the laptop and the CodeBuild image can also cause test failures, so I pin those versions in the `buildspec`. Once I identify the difference, I make the build environment reproducible, often by adding a Docker-based build or by running the same `buildspec` in a local CodeBuild agent.

---

### **S19. After scaling out, the application runs out of database connections and starts failing. How do you fix it?**

**Answer:** Running out of connections usually means the total pool size across all instances exceeds the database's maximum connection limit. I calculate the expected total connections by multiplying the per-instance HikariCP maximum pool size by the number of instances and compare that to the RDS `max_connections` setting. If the total is close to or above the limit, I reduce the per-instance pool size, because a larger pool on each instance does not help if the database cannot accept them. I also check for connection leaks caused by missing `try-with-resources` or long-running queries that hold connections. For read-heavy workloads, I add read replicas and route read traffic to them, which both scales capacity and reduces pressure on the primary. If connection demand is genuinely high, I scale the RDS instance class vertically to support more connections or use RDS Proxy to pool and share database connections efficiently. I avoid simply raising the pool size as the first fix, because that usually makes the problem worse.

---

**S20. A production secret in AWS Secrets Manager is rotated, and the application stops working. How do you handle secret rotation without downtime?**

**Answer:** The problem is likely that the application reads the secret only at startup and never refreshes it, so when the secret value changes the cached credentials become invalid. I first restore service by restarting the affected instances or rolling back the secret rotation if possible. For a proper fix, I change the application to refresh the secret periodically or on authentication failure, using the AWS SDK to retrieve the latest value from Secrets Manager. When rotating database credentials, I use the Secrets Manager rotation feature that stores both the current and pending versions during the rotation window, so the application has time to pick up the new value while the old value remains valid. I also make sure the database user is not dropped immediately after rotation, because open connections from the old credential must finish their work. Ideally, the application uses IAM database authentication or RDS Proxy so password rotation is handled outside the application entirely. The key is that secret rotation should be graceful, not an all-at-once event.